Proxy + Bot Account Architecture

LikeSee.AI — Instagram Activity Tracking *Technical brief for product / client-relations stakeholders Date: 2026-05-08*

Why this document exists

The question came up: *"I thought the proxy was meant to prevent detection at all — clearly I was off."*

That is a very common (and reasonable) assumption. This document explains what proxies actually do, what else has to happen for the system to look human to Instagram, and the layered defense strategy LikeSee.AI uses.

TL;DR

A proxy hides **one thing**: the IP address Instagram sees.

That alone is not enough. Instagram has at least **eight independent detection signals** beyond IP. The proxy makes us look like we are coming from a residential household in the right country — but the rest of the disguise (browser fingerprint, device identity, behavior pattern, account history) has to be built separately.

When all eight layers line up, we look like a real person. When any one of them is wrong, Instagram can flag the account even on a perfect proxy.

What a proxy does

```
                [ LikeSee.AI server ]
                          |
                          v
        [ Smartproxy residential IP ]    ← what Instagram sees
                          |
                          v
                  [ Instagram.com ]
```

A proxy is a relay. Our server sends the request to a proxy, and the proxy forwards it to Instagram. From Instagram's perspective, the request came from the proxy's IP, not ours.

That is the entire job of the proxy.

The IPs we use are **residential proxies** — meaning they belong to real home internet connections in the US, UK, Germany, and France. This matters because Instagram knows which IP ranges belong to data centers (AWS, Hetzner, DigitalOcean, etc.) and treats traffic from those IPs as "automation by default." Residential IPs blend in with real users.

What proxies *do not* do

- They do not change what your browser looks like
- They do not change how your account behaves
- They do not bypass two-factor authentication
- They do not make a brand-new account look established
- They do not slow down clicks to look human
- They do not change your account's history or friend graph

These are all separate problems with separate solutions.

Instagram's detection stack

Instagram uses many signals in parallel. A proxy only addresses signal #1:

#	Signal	What it checks
1	**IP / ASN**	Is this a datacenter? A flagged proxy pool? Wrong country for the account?
2	**Browser fingerprint**	Canvas hash, WebGL hash, fonts, screen size, timezone, language
3	**Device fingerprint**	User-Agent string, GPU, audio context, hardware concurrency
4	**TLS fingerprint**	The exact "shape" of your TLS handshake (JA3) — automation libraries have distinctive ones
5	**Behavior pattern**	Click timing, mouse movement, scroll pattern, dwell time on pages
6	**Account history**	Account age, posting cadence, friend graph, prior IP locations

7	**Session reuse**	Are cookies / login tokens reused in a way real users do?
8	**Challenge response**	If we ask for an email / SMS / 2FA code, do you actually get one?

A proxy only addresses signal #1. The other seven are independent.

The layered defense LikeSee.AI uses

Layer 1 — Per-bot dedicated sticky proxy

Each bot account gets **its own** sticky residential IP that does not change. If bot `amaliapaul37` always logs in from `203.0.113.42` and one day shows up from a different IP, Instagram treats that as suspicious. So we keep one proxy per bot for as long as the proxy stays alive.

Distribution across 25 bots: **15 US**, **8 UK**, **3 Germany**, **2 France** — to match the locale claims of each account.

Layer 2 — Per-bot device fingerprint

Each bot has a unique combination of:

- User-Agent string (Chrome on Windows, Chrome on Mac, Firefox on Linux, etc.)
- Viewport size (1366×768, 1920×1080, 1440×900, etc.)
- Timezone (`America/New_York` , `Europe/London` , `Europe/Berlin` , etc.)
- Locale (`en-US` , `en-GB` , `de-DE` , etc.)
- Awake-hours window (e.g., 10:00–23:00 local time only)

These are stored on the bot's row in the database and replayed every time that bot logs in. If a different bot used the same fingerprint, they would look related to Instagram — so each is unique.

Layer 3 — Account tier system

Bots are placed into tiers based on how much we trust them:

- **canary** (2 bots): used first, for low-risk read-only work. If they burn, we lose the canaries — not the workers.
- **worker** (8 bots): the daily drivers. Used after canaries have proven the proxies and login flow are healthy.
- **reserve** (15 bots): held back. Used only when workers fall out of the pool. Keeps a stockpile alive.
- **retired**: bots that have been banned or burned. Never reused.

This means a single bad day can take out 1–2 canaries without disturbing the rest of the pool.

Layer 4 — Three-strikes per proxy endpoint

Smartproxy's residential pool is not 100% healthy. Some IPs are rate-limited or already on Instagram's blocklist. Each IP is given **three chances** — if it fails a login three times, it is auto-disabled for 24 hours and the bot is rotated to a fresh IP.

Layer 5 — Daily call budget per bot

Each bot has a hard cap on how many Instagram requests it can make per day. Once it hits the cap, it sleeps until tomorrow. This prevents a single misconfigured worker from running a bot 1,000 times in an hour.

Layer 6 — TOTP / Email-OTP for challenges

When Instagram presents a verification challenge (email code, SMS code, 2FA app), we have:

- **TOTP** (Google Authenticator-style 2FA secret) on bots configured for it. Generates the 6-digit code locally — no network round trip.
- **Email-OTP fallback** for bots without TOTP. We poll the bot's mailbox via IMAP (rambler.ru, onet.pl) or the tmail.gg web inbox API and read the verification email automatically.

If neither is available, the login fails fast with a clear error rather than burning retries.

Layer 7 — Warming period

Brand-new bot accounts have **zero account history** — which is itself a red flag to Instagram. We ramp new bots up gradually: day-1 only logs in, day-2 makes a few profile views, day-3 follows one or two accounts, etc. By day-7 the bot has enough activity history to look like a real user who occasionally browses.

Layer 8 — Health probe before assignment

Before assigning a proxy to a bot, we probe it against `instagram.com/favicon.ico` (not just a generic IP service). This catches proxies that are *specifically* blocklisted by Instagram while still working for other sites — preventing wasted login attempts on a dead-on- arrival IP.

Why bots can still get banned

Even with all eight layers in place, Instagram can still flag accounts for reasons we cannot easily control:

1. **Account purchase history** — if a batch of accounts was bought from a vendor that Instagram has linked together, they can all get banned at once when the vendor is identified.
2. **Login geography mismatch** — if an account was originally created in Indonesia and we only ever log in from the US, that pattern alone is a yellow flag.
3. **Burst traffic across the pool** — if 20 bots all log in within 2 minutes of each other from related IP ranges, Instagram correlates them.
4. **Captcha / "we sent your code" loops** — once Instagram is suspicious, it will keep asking for verification codes. Each failed attempt makes the account look more bot-like.
5. **TLS / HTTP fingerprint of the automation library** — Playwright (the browser-automation library we use) has a slightly unusual TLS signature. Mitigated but not invisible.

What happened with the recent bot batch

The first batch of bots (7 accounts from December) got flagged because **all seven were brand-new, all logged in within the same hour, and all used the same datacenter IP range** before the per-bot proxy system was in place. Instagram correlated them by IP proximity and behavior pattern, and shadow-banned the lot.

Lessons that fed into the v2 system (May 2026):

- Per-bot **dedicated sticky proxy** so no two bots share an IP
- **Geographic distribution** matching the claimed locale
- **Tier system** so we never burn canaries and workers together
- **Slow drip rollout** — 1 canary first, observe 12–24 hours, then promote workers gradually

The v2 batch (25 bots) was seeded with all of these in place. As of this writing the v2 batch has **not been banned** — but a separate proxy-pool quality issue meant logins were failing at the *connection* layer (Instagram never even saw most attempts). That issue is now mitigated by the IG-specific health probe in Layer 8 above.

Recommendations for the client conversation

If the client asks "why do we need proxies *and* bots?", the short answer:

> *The proxy hides where we are coming from. The bot accounts are the identity Instagram sees. Both are needed — a proxy without a real-looking account looks like a stranger walking into a bank with a mask, and a real account without a proxy looks like a real person logging in from a known automation server. Neither alone is enough.*

If they ask "can the bots get banned?", the honest answer:

> *Yes, occasionally — Instagram is actively defensive and any individual account can be flagged. The system is designed so a single ban does not take down the service: when one bot fails, the next one in the pool takes over automatically. We keep canaries to catch problems early, and a reserve pool of bots ready to swap in.*

If they ask about **scale**:

> *We have 25 active bot identities. Each can do up to a few hundred Instagram operations per day, giving the platform headroom for several thousand tracked accounts before we need to expand the pool.*

Summary

Question	Short answer
Does the proxy prevent detection?	No — it only hides our server's IP.
What does prevent detection?	Eight layers working together (proxy + fingerprint + behavior + history + …).
Will bots get banned?	Some will. The pool is sized to absorb that.
What is the canary system?	First-line bots used to detect problems before they hit the rest of the pool.
Can we scale?	Yes — adding bots is a config + seeding step, not a code change.

For technical questions, contact the engineering team.